

Automation with Selenium

Assignment 5: The HTML Web Table Extractor

1. Problem Statement: -

Find a page containing a multi-column dynamic data table. Write a script to iterate through rows and columns. Locate a specific row by matching a name string, and retrieve the corresponding value from the Status/Price column next to it.

2. Objective: -

To extract and process data from complex HTML table structures dynamically.

3. Tools, Software, and Concepts Used: -

Python 3.x, Selenium WebDriver, XPATH.

4. Source Code / Implementation: -

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
driver = webdriver.Chrome()
driver.maximize_window()
driver.get("https://rahulshettyacademy.com/AutomationPractice/")
time.sleep(3)
table_rows = driver.find_elements(By.XPATH,
    "//table[@name='courses']/tbody/tr")
search_name = "Master Selenium Automation in simple Python Language"
price = None
for row in table_rows[1:]:
    columns = row.find_elements(By.TAG_NAME, "td")
    if len(columns) > 1 and columns[1].text == search_name:
        price = columns[2].text
        driver.execute_script("arguments[0].scrollIntoView({block:
'center'});", row)
        driver.execute_script("arguments[0].style.backgroundColor =
'yellow';", row)
        break
print(f"The price for '{search_name}' is {price}")
time.sleep(10)
driver.quit()
```

5. Output & Screenshots: -

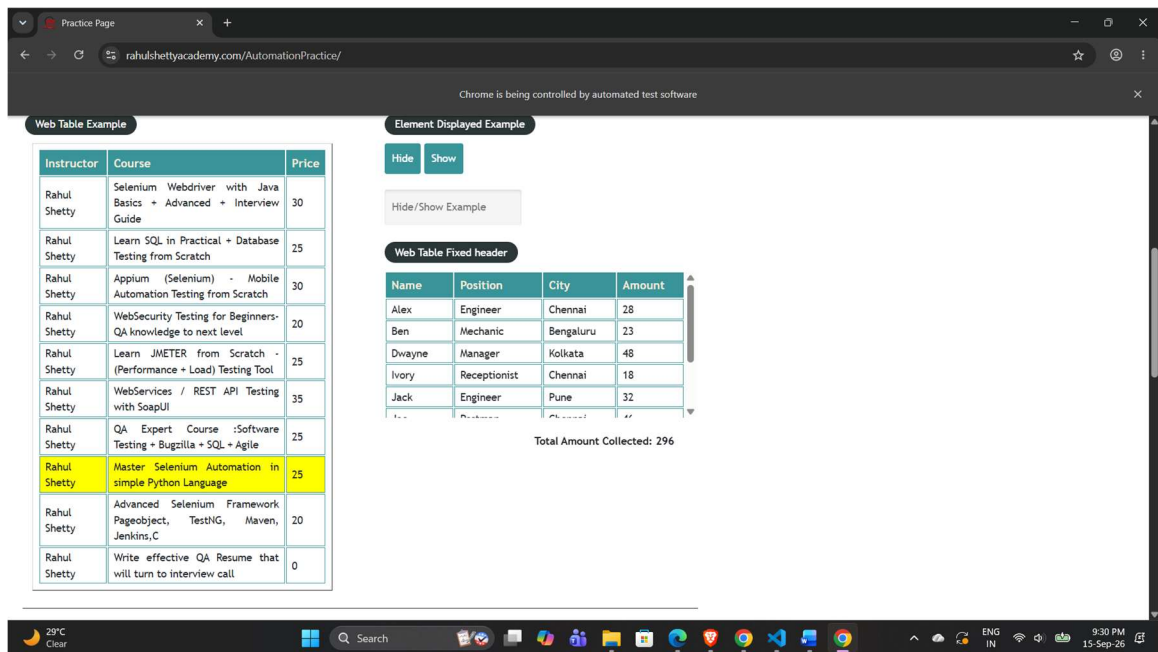


Fig.1: Selenium successfully locating and highlighting the target row in the web table

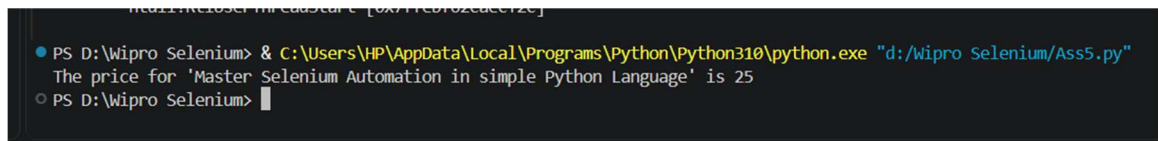


Fig.2: Terminal output displaying the extracted price

6. Result, Observation, and Conclusion: -

- Result: The script dynamically traversed the table and extracted the correct corresponding data.
- Observation: By identifying the row context first, finding sibling columns becomes highly reliable compared to absolute XPath indexing.
- Conclusion: Web table parsing is crucial for data validation and involves strategic looping and element-in-element searching.